GePNIC SASCI Secured Data Sharing

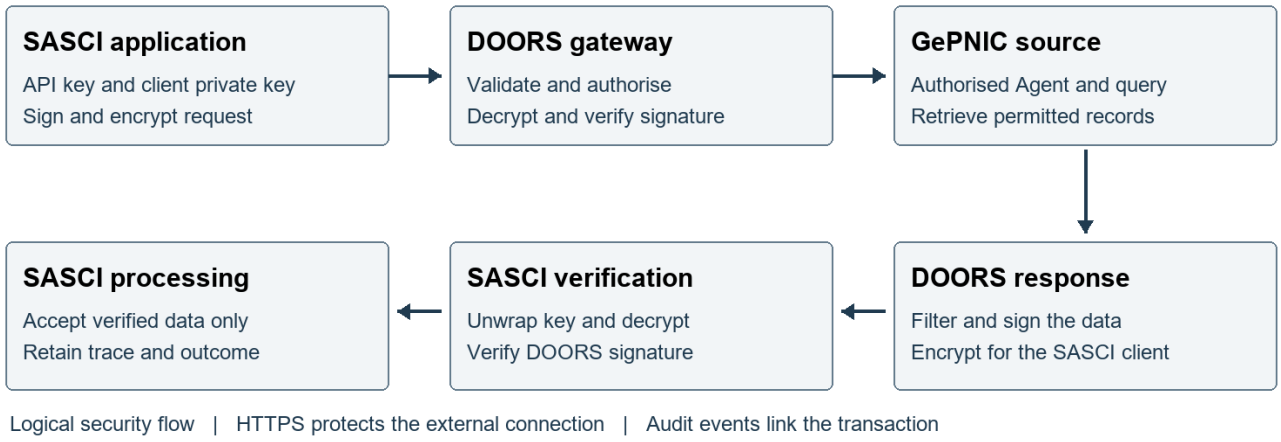
Current DOORS implementation and security evidence

Prepared for the SASCI Integration Audit Team
Document version 1.0 | 24 September 2026

Purpose and conclusion

DOORS provides a controlled data-sharing path in which the SASCI application authenticates with an API key, signs and encrypts its request, and receives data encrypted for its registered cryptographic identity. DOORS authorises the query and source access before execution. The SASCI application decrypts the response and verifies the DOORS signature before consuming the returned data.

This document explains the current protocol 2 implementation, its trust boundaries and the evidence supporting each control. It covers the complete request and response lifecycle for the onboarded AOC Details, Financial Evaluation Details and Tender Status services. The separate GePNIC SASCI Integration document remains the reference for dataset fields and sample business responses.



Evidence basis

The implementation statements are traceable to the current gateway, SDK generator, access-policy and audit sources listed in section 11. Twenty-eight targeted local tests passed on 24 September 2026. These results demonstrate the tested application controls; the deployment evidence in section 9 establishes their application to the actual SASCI client, endpoints and transactions.

The secured profile described here requires protocol 2 to be enabled for each assigned query, response encryption to be enabled for the client, registered key material, and approved HTTPS endpoints. This document records implementation evidence and audit acceptance criteria; it does not constitute a production security certification.

2 Client onboarding and key ownership

Before a SASCI application can exchange data, the responsible DOORS administrator registers an API client, assigns its permitted queries and source Agents, and configures its access policies. SASCI installs the client-specific SDK and supplies the production cryptographic identity. These registrations bind a credential to an authorised data-sharing scope. [S1 S3 S5]

AES is the symmetric cipher used for payload encryption. RSA is the public and private key mechanism used for key wrapping and signatures. PKCS12 is the client identity container. JWKS is the JSON Web Key Set used to publish the DOORS public key. An IV is the per-message initialisation value used by the payload cipher.

Asset	Owner and use	Audit evidence
API key	Issued for the DOORS client. Sent as X-API-KEY on machine requests; identifies the client for policy checks.	Client ID and masked key reference; active status.
SASCI private key	Held by SASCI in its PKCS12 identity. Signs requests and unwraps response session keys.	Public-key fingerprint, custody owner and access controls; never the private key.
SASCI public key	Registered against the DOORS API client. Verifies requests and wraps response session keys.	Registered fingerprint compared with the installed SDK identity.
DOORS private key	Resolved from the active DOORS key record. Unwraps request session keys and signs responses.	Active key ID, public fingerprint and restricted key-store access record.
DOORS public key	Published through the gateway JWKS endpoint. Used for request key wrapping and response signature verification.	Trusted HTTPS endpoint, current public key and key ID.
AES session keys	Generated separately for requests and responses. Protect the application payloads for that message.	Algorithm and round-trip verification; no key values in the audit packet.

Client identity registration

The production SDK loads its private key and certificate from a local PKCS12 file using the configured wallet path, alias and password. Only the public certificate or public key is registered with DOORS. The SDK derives a SHA-256 fingerprint from the encoded public key for identification and troubleshooting. API-key possession does not replace possession of the registered private key. [S1]

Handshake verification

The SDK sends its public key to POST /api/v1/master/gateway/handshake with X-API-KEY. DOORS resolves the active client, parses the supplied RSA key and compares its encoded bytes with the registered public key. An unregistered or mismatched key is rejected. An accepted handshake caches the matching public key; it does not upload the private key or prove private-key possession. The subsequent request signature provides that proof. [S2]

3 Request preparation and encryption

The current SDK prepares the following logical request fields for the selected API. The signed clientId and query bind the protected request to the registered client and URL route. agentIds and params define the authorised data requirement. The requestId is a new UUID and issuedAt is the current UTC time. [S1 S4]

Field	Purpose
protocolVersion	Integer 2 in both the protected JSON and outer envelope.
clientId and query	Client identifier and selected registered query name; checked against the authenticated client and route.
requestId	Canonical lowercase UUID identifying this request attempt.
issuedAt	UTC ISO instant used for the freshness check.
agentIds	Requested source Agent identifiers; requesting ALL remains limited to authorised Agents.
params	Parameters required by the selected query contract, such as its date input.

Protection sequence in the SDK

1 Serialize once. Create the exact UTF-8 JSON bytes containing all protected fields. Signing and encryption use these same bytes. Whitespace or field-order changes after signing change the signed message.

2 Generate fresh material. Generate a random 256-bit AES key and a random 12-byte IV for this request.

3 Encrypt the payload. Use AES/GCM/NoPadding with a 128-bit authentication tag. The encoded secureData value contains ciphertext followed by the GCM tag.

4 Wrap the AES key. Encrypt the AES key with the DOORS RSA public key using RSA/ECB/PKCS1Padding. The client private key is not used for this encryption.

5 Sign the request. Use SHA512withRSA and the SASCI private key to sign the same plaintext JSON bytes. The signature is Base64 encoded.

6 Send the envelope. POST to /api/v1/master/gateway/orchestrate/{query} over the approved HTTPS endpoint, using Content-Type: application/json and X-API-KEY.

```
{"protocolVersion": 2,
 "encryptedKey": "<Base64 RSA wrapped AES key>",
 "iv": "<Base64 12 byte IV>",
 "secureData": "<Base64 GCM ciphertext and tag>",
 "signature": "<Base64 client signature>"}
```

The envelope above is a field illustration, not a recorded SASCI transaction. Base64 transports binary values; it is not encryption. The API key remains an HTTP header protected by HTTPS, outside the signed and encrypted JSON body.

4 Gateway validation before execution

The machine-route subscription interceptor first requires a valid client API key and evaluates service and time-access policies. The API-key interceptor then checks active status and any configured source-IP allowlist. A portal login session cannot substitute for the client credential on these machine routes. [S3 S6]

Authorisation and cryptographic checks

Client and query scope. The gateway requires an active client, an active APPROVED query template and an explicit client-to-query mapping. It rejects malformed response-filter policy. It resolves the client public key from the verified cache or registered profile. [S2 S5]

Protocol permission. Protocol 2 must be enabled for that client and that query, enabled in the protocol catalogue and not retired. An unsupported or disabled selection is rejected. [S4]

Envelope validation. encryptedKey, iv, secureData and signature must be present and nonblank. DOORS unwraps the AES key using its active RSA private key and requires a 32-byte key and 12-byte IV. [S2]

Payload integrity and sender identity. DOORS decrypts with AES-GCM and checks its authentication tag. It then verifies the client signature against the registered public key, using SHA512withRSA, with SHA256withRSA also accepted by the current verifier. Invalid crypto prevents the request from reaching report execution. [S2]

Signed field binding. Only after decryption and signature verification does DOORS parse the protected JSON and compare protocolVersion, clientId and query with the outer request and authenticated route. [S2 S4]

Freshness and duplicate protection

Check	Current behaviour
Clock reference	Database clock_timestamp() provides the validation time.
Maximum age	issuedAt must not be more than 300 seconds in the past.
Future tolerance	issuedAt must not be more than 30 seconds in the future.
Duplicate identity	The pair of client ID and request UUID is claimed atomically in gateway_request_claim.
Claim lifetime	The claim expires at issuedAt plus 360 seconds; expired claims are cleaned up.
Failure and retry	The claim commits in a separate transaction and survives a downstream failure. A repeated claim is rejected with HTTP 409.

These controls are checked before the query executes. After an uncertain timeout, reconcile the request with the DOORS trace and audit outcome before creating a fresh request ID; duplicate rejection is not a cached-response or automatic retry mechanism. [S4]

5 Governed retrieval and response protection

Retrieve only the permitted data

DOORS resolves the requested source Agents from the intersection of the caller and query permissions. If ALL or no Agent is requested, only the authorised set is selected. An explicitly requested Agent outside that set is rejected. The selected Agent must also be active. [S7]

The report service verifies that the selected SQL template is active, approved and read-only, checks input parameters and dispatches the server-held template with named parameters to the registered Agent. SASCI does not supply SQL or database credentials through the external orchestration request. The Agent executes against its configured GePNIC source using the supplied governed connection context. [S7 S12]

The gateway normalises results into the response contract and applies any client-specific response filter before observation, serialisation, signing or encryption. A configured field that is absent from the returned data produces an empty result rather than unfiltered disclosure. Without a configured response filter, this additional row restriction is not applied. [S2 S8]

Create the protected response

1 Form the result. Serialise the final status, message and payload into UTF-8 JSON after the permission and segregation steps.

2 Sign the result. DOORS signs those JSON bytes with its active private key using SHA256withRSA. This signature authenticates the returned plaintext when verified with the trusted DOORS public key.

3 Encrypt the result. Generate a new 256-bit AES key and random 16-byte IV. Encrypt the JSON with AES/CBC/PKCS5Padding.

4 Wrap for the intended recipient. Encrypt the response AES key with the registered SASCI RSA public key using RSA/ECB/PKCS1Padding. The corresponding SASCI private key is required to open it.

5 Return the envelope. Return encryptedKey, iv, secureData and signature. X-Content-Secure is true, X-DOORS-KID identifies the active DOORS key and X-DOORS-TRACE identifies the server-side transaction. [S2]

Property	Request	Response
Payload cipher	AES-256-GCM	AES-256-CBC with PKCS5 padding
IV length	12 bytes	16 bytes
Signature created by	SASCI using SHA512withRSA	DOORS using SHA256withRSA
RSA key wrapping	DOORS public key	SASCI public key
RSA transformation	RSA/ECB/PKCS1Padding	RSA/ECB/PKCS1Padding

The response profile is controlled by isEncryptionEnabled. The secured-sharing evidence must show this setting enabled for the SASCI client; the existence of encryption code alone does not establish that a particular response was encrypted.

6 SASCI decryption and signature verification

Receiving HTTP 200 is only the transport and server-processing outcome. The SDK must complete local cryptographic processing before passing the data to the SASCI application. The current generated client follows the sequence below for an encrypted response. [S1]

1 Capture the response context. Read the response envelope, X-DOORS-KID and X-DOORS-TRACE. Preserve the trace for correlation with DOORS. The secured profile should receive X-Content-Secure: true and all four envelope fields.

2 Unwrap the response key. Base64-decode encryptedKey and decrypt it with the SASCI private key from the installed PKCS12 identity, using RSA/ECB/PKCS1Padding. A different private key cannot correctly recover the response AES key.

3 Decrypt the payload. Base64-decode iv and secureData. Use the recovered AES key, the 16-byte IV and AES/CBC/PKCS5Padding to recover the UTF-8 JSON bytes. Treat these bytes as untrusted until the signature is verified.

4 Resolve the DOORS public key. Retrieve or use the cached public key from the trusted gateway JWKS endpoint. Associate the verification evidence with the response key ID and the actual public-key fingerprint used.

5 Verify the signature. Use SHA256withRSA to verify signature over the exact decrypted JSON bytes. Do not parse and reserialise the JSON before checking it: whitespace and ordering changes can invalidate the signature.

6 Release verified data. Return the decrypted JSON to the calling application only after signature verification succeeds. Check the application status and expected schema before importing records into SASCI.

What the client rejects

The generated client throws an exception on RSA key-unwrapping errors, AES decryption errors or failed DOORS signature verification. It classifies the failure stage, records the trace and public-key fingerprint, and attempts to report the failure to the DOORS telemetry endpoint. A telemetry transport failure does not suppress the original local cryptographic exception. [S1 S9]

Confidentiality and authenticity are separate checks

Response encryption restricts recovery of the data to the holder of the recipient private key. The digital signature establishes integrity and possession of the trusted DOORS signing key. Decryption success alone does not authenticate the response. Neither check establishes the business correctness of the source records; that remains subject to the API contract and source-system controls.

Application handling after verification

The receiving team should restrict access to decrypted output, avoid putting data and secrets into console or support logs, and retain only the information required for the approved integration purpose. If the SDK runner prints successful JSON output, redirect it only to an authorised destination with defined retention. These receiving-system controls form part of the deployment evidence, rather than being supplied by the cryptographic envelope itself.

7 Access governance and operating controls

Suspension and permitted hours

An inactive API client is rejected. A manually disabled service for a mapped GePNIC Agent also prevents machine access. The subscription interceptor re-evaluates policy on each covered request, including handshake and telemetry calls, rather than treating a previously accepted handshake as continuing authorisation. [S3 S6]

DOORS supports explicit date-and-time access windows and subscriber weekly time slots. Weekly evaluation uses Asia/Kolkata. An empty calendar means no calendar restriction; unrestricted weekly mode permits all hours. A restricted weekly schedule with no permitted slot denies access. License dates and payment balances are recorded but do not automatically suspend service in the current policy. [S6]

Source IP and transport trust

A configured client IP allowlist is enforced. An empty allowlist adds no IP restriction. Trusted-proxy configuration determines when forwarded client-IP headers are accepted; untrusted forwarding headers are removed. The deployment must preserve the intended proxy boundary and the real client address. [S3]

HTTPS is required for the secured deployment, because the API key is carried in a header and JWKS is used to establish the DOORS public key. The generated production runner configures a PKCS12 truststore. The selected gateway URL, certificate chain, hostname validation and allowed TLS settings must be evidenced on the actual endpoint. This application exchange does not by itself demonstrate mutual TLS. [S1]

Key custody and rotation

SASCI retains its private key and wallet credentials locally. DOORS uses the active certificate-registry entry to load its private key from the configured key-storage directory. File permissions, backups and access monitoring therefore matter; the reviewed loader does not establish hardware security-module custody. Separate production identities from test packages and restrict all credential-bearing SDK downloads. [S1 S10]

For a client-key change, register the corresponding public key and compare its fingerprint with the newly loaded SDK identity before resuming calls. For a DOORS-key change, capture the new active key ID, publish the public key, refresh the client key cache and verify a controlled request and response. Retain the change authorisation and outcomes as rotation evidence.

The internal trust boundary

DOORS decrypts requests and handles source results in its own process. The external cryptographic envelope is therefore application-to-gateway protection, not encryption that keeps data hidden from DOORS. Master-to-Agent calls use registered endpoint URLs and a separate connection path. The database password is wrapped using the configured shared transit secret; the reviewed helper derives a 128-bit AES key from its first 16 UTF-8 bytes and uses AES/ECB/PKCS5Padding. This protects that credential field, not the whole Agent exchange. Internal HTTPS, network restrictions, Agent access controls and database TLS and privileges require their own deployment evidence. [S7 S11 S12]

8 Audit trail and failure handling

The gateway assigns X-DOORS-TRACE to the orchestration transaction. Audit records include the client identity, query, endpoint, method, status, source IP, duration, record count and trace. The generated command representation masks X-API-KEY. The protocol request UUID and the DOORS trace serve different purposes: retain both when assembling transaction evidence. [S2 S4 S9]

Condition	Current response or handling	Evidence to retain
Missing or invalid API key	HTTP 401 on machine access.	Masked request, response code and trace if supplied.
Disallowed IP or suspended access	HTTP 403; policy-specific error where provided.	Policy snapshot and rejected transaction.
Unregistered or mismatched client key	Handshake conflict with key-registration or key-mismatch code.	Public fingerprints and handshake outcome.
Malformed envelope or tampered GCM	Crypto rejection; tampered GCM returns HTTP 400 with DOORS-CRYPTO-INVALID.	Negative-test response and absence of query execution.
Expired timestamp or invalid binding	Request rejected before execution; freshness errors use HTTP 400.	Signed metadata and rejection outcome.
Duplicate client request UUID	HTTP 409 with DOORS-REQUEST-REPLAY.	First outcome, replay outcome and claim evidence.
Unauthorised query or Agent	Execution is denied.	Assigned scope and rejection outcome.
Client cannot decrypt or verify	SDK raises a cryptographic exception and attempts telemetry.	Client failure stage, trace and related DOORS event.

Correlating server and recipient outcomes

The client telemetry endpoint accepts failure details for the client owning the trace and updates the matching audit record. It is a client-reported diagnostic, not an independent proof of receipt. A server success record does not prove that SASCI successfully decrypted or accepted the data. Pair it with a recorded SDK verification outcome and the receiving application acknowledgement. [S9]

Evidence protection

Export audit rows through an authorised process and record the extraction time, environment and release. Redact API keys, passwords, private keys and unnecessary business data before sharing. Hash each evidence file and preserve the original in a controlled repository. The implementation shown here uses database audit records; it does not itself establish immutable storage, a retention period or legal non-repudiation. Those controls need operating evidence.

9 Evidence required for secured sharing

Use this matrix to assemble proof for the actual SASCI deployment. Each item links a claimed protection to an observable configuration or transaction artifact. Keep identifiers consistent across the packet and avoid including production secrets.

ID and control	Artifact for the audit team	What it establishes
E01 Client scope	Dated client profile showing client ID, active status, masked key reference, assigned APIs and Agents.	The credential is bound to an approved access scope.
E02 Secure profile	Protocol 2 policy for each assigned query and response-encryption setting.	The described request and response protections are enabled.
E03 Key registration	SDK public-key fingerprint, registered fingerprint, DOORS key ID and public-key fingerprint; handshake result.	The parties use the intended identities.
E04 Transport	Approved URL, certificate and trust-chain validation, TLS configuration, source-IP and proxy policy.	Header credentials and public-key discovery use the trusted connection.
E05 Protected request	Sanitised envelope, protected metadata record, timestamp and request UUID.	A specific request was signed, encrypted and bound to the intended client and query.
E06 Protected response	Ciphertext envelope with X-Content-Secure, X-DOORS-KID and X-DOORS-TRACE.	A specific result used the encrypted response profile.
E07 Recipient verification	SDK outcome for RSA unwrap, AES decryption and signature verification; controlled receipt acknowledgement.	The intended recipient successfully verified that response.
E08 Rejection evidence	Tampered, stale, duplicate, wrong-scope and suspended-access test outcomes.	The tested controls deny invalid access or data.
E09 Internal safeguards	Agent endpoint and network controls, database account grants and transport protection, key-file permissions.	The internal processing boundary is protected.
E10 Audit and custody	Correlated audit extract, evidence hashes, extraction details, retention and access records.	The proof can be traced and preserved.

Transaction evidence record

For each of the three SASCI services, record the environment, gateway release, SDK release and file hash, client ID, query, source Agents, request UUID, issuedAt, HTTP status, DOORS trace, DOORS key ID, client fingerprint, verification result and received record count. Link the server event and recipient acknowledgement using the trace. Store plaintext samples separately under the approved data-handling controls.

10 Verification results and audit acceptance

The following existing automated tests were executed against the current local working tree on 24 September 2026. All 28 tests passed with zero failures, zero errors and zero skips. The tests use controlled fixtures and, where applicable, mocked collaborators; they are not a live SASCI-to-GePNIC production execution.

Test suite	Tests	Control exercised
GatewayCryptoValidationTest	1	Tampered AES-GCM payload rejected with generic crypto error before report execution.
GatewayProtocolServiceTest	7	Duplicate claim, freshness, signed bindings, per-query protocol permission and unsupported-handler rejection; includes policy compatibility checks.
ApiSubscriptionInterceptorTest	3	Machine-route enforcement, missing-key rejection despite session authentication, route inventory.
ApiLicensePolicyTest	6	Manual suspension, calendar boundaries and the advisory treatment of license dates and balances.
ApiWeeklyAccessPolicyTest	5	India-time windows, overnight and week boundaries, closed slots and validation.
ClientSpecificDataSegregationServiceTest	6	Authenticated-client filtering, missing-field empty result, invalid policy rejection and unconfigured-filter behaviour.
Total	28	Zero failures, errors or skips in the completed run.

Execution used Maven Surefire with Java 24.0.1 and the Byte Buddy experimental compatibility flag required by the existing Mockito instrumentation. The authoritative result files are the six matching TEST XML reports in masterapi/target/surefire-reports. [T1]

Witnessed verification for the deployment

Successful protected exchange. For each assigned SASCI API, use an approved test date and dataset, capture E01 through E07, and confirm the decrypted response matches the intended source scope. Record the current endpoint and release.

Cryptographic rejection. Alter ciphertext or a signature in a controlled test; verify rejection before data use. Attempt response verification with an incorrect DOORS public key and confirm the client fails. These response-side negative tests are acceptance actions, not results of the 28-test run above.

Access and replay rejection. Repeat an accepted request UUID, submit stale metadata, use an unassigned query or Agent, and exercise a configured suspension or closed access window. Record the server and recipient outcomes.

Close the evidence record. The DOORS operator and SASCI recipient record the evidence references, date, environment and observed results. The audit team assesses the packet against its acceptance criteria. No production approval or witness signature is implied by this document.

11 Source traceability and assurance boundaries

Source identifiers in the preceding sections refer to the current implementation below. File names are relative to the stated project. Method names provide stable review anchors. The snapshot is the local working tree on 24 September 2026; it includes uncommitted changes and is not represented as a deployed release.

S1 doors-ui ApiClientManagement.vue Generated production SDK: loadPermanentIdentity, executeHandshake, encryptPayloadData, generateDigitalSignature, fetchReportData and telemetry.

S2 masterapi ExternalGatewayController.java establishCryptographicHandshake, proxyOrchestration and getPublicJwks.

S3 masterapi ApiKeyInterceptor.java preHandle; with WebConfig, MachineApiRoutes and TrustedProxyConfiguration in the same package.

S4 masterapi GatewayProtocolService.java requireAllowed, validateAndClaim and updatePolicy.

S5 masterapi ApiClientExecutionPolicy.java authorize; ApiClient entity supplies isEncryptionEnabled and client key registration fields.

S6 masterapi ApiSubscriptionService.java access and enforce; ApiLicensePolicy, ApiWeeklyAccessPolicy and config/ApiSubscriptionInterceptor.

S7 masterapi ReportViewerService.java executeReport and resolveAgents; SQL template, parameter and Agent checks.

S8 masterapi ClientSpecificDataSegregationService.java apply and recursive filter before outbound cryptographic protection.

S9 masterapi AuditAspect.java Audit persistence and generateCurl; ExternalGatewayController.captureClientTelemetryFault.

S10 masterapi CertificateStorageService.java getActiveSigningPrivateKey; active registry pointer and file-backed private key.

S11 masterapi AgentTransitEncryption.java encryptPassword; masterapi util/EncryptionUtils supplies the credential-field transformation.

S12 agent-api AgentQueryController.java executeProductionQuery and connection construction; util/EncryptionUtils decrypts the transit credential.

T1 Test evidence: the six named suites in section 10 and their Surefire XML reports. Source snapshots: doors-ui HEAD 1dbe7e0 and masterapi HEAD acacf3e, plus the reviewed working-tree content. Evidence references should be replaced or supplemented with the deployed release and dated artifacts when recording a live transaction.

12 Conditions for an accurate security assertion

The current implementation supports the protected flow described in this document. The following boundaries determine what an audit conclusion can reasonably assert and what must be demonstrated through deployment evidence.

Secure mode is an explicit configuration

The request must use the enabled protocol 2 policy for its client and query. Response encryption is a separate client setting. The generated SDK currently returns a response unchanged when secureData is absent; therefore an HTTP success or SDK return by itself does not prove secured sharing. E02 and E06 must demonstrate the encrypted response profile, and the integration acceptance check should reject an unexpected unprotected business response. [S1 S2 S4]

Public key discovery needs trusted transport and rotation evidence

The current JWKS endpoint publishes the active public signing key. The generated SDK extracts its RSA modulus and exponent and caches the key under the requested key ID; it does not independently compare the returned JWKS kid with that requested ID. Record the actual public-key fingerprint and test cache refresh during rotation rather than treating the response key-ID header alone as proof of key selection. [S1 S2]

Certificate parsing is not certificate lifecycle validation

The public-key parser accepts RSA public keys and X.509 certificates and extracts their public keys. The handshake verifies equality with the registered key. These actions do not themselves establish certificate-chain, expiry or revocation checks on that application identity. Preserve the identity approval and certificate lifecycle evidence separately. TLS server certificate validation is a separate connection control. [S1 S2]

Signatures protect their defined message

Request signatures cover the protected request JSON, including protocol, client, query, UUID, time, Agents and parameters. Response signatures cover the returned plaintext JSON. The response does not cryptographically echo the protocol request UUID or timestamp in the reviewed envelope, and the response headers are outside that signature. Correlate the response with the intended call using trusted transport, the server trace and recipient records; do not claim response replay protection solely from request replay checks. [S1 S2 S4]

Audit completeness requires operating evidence

Audit database writes and client telemetry are useful for investigation, but the reviewed paths log audit-write failures and allow telemetry failures without replacing the primary transaction outcome. Monitor audit persistence and preserve server and recipient records. Neither the source review nor the local fixture tests establishes immutable audit storage, continuous control operation, production TLS posture or internal network isolation. [S9]

Audit conclusion supported by this document

The reviewed implementation provides request confidentiality and integrity checks, registered-key request authentication, scoped execution, request freshness and duplicate rejection, and recipient-targeted response encryption with DOORS signature verification. The completed local tests support the specific controls listed in section 10. A deployment-specific secured-sharing conclusion is supported when the evidence in section 9 demonstrates that these controls were enabled and successfully exercised for the named SASCI services.